번호 : /

강원대학교 시험 답안지

			감독교수확인	이 두 호
2025 학년도 2학기 기말고사	시험과목 : 머신러닝2 (3반)	전공 : AI소프트웨어학과 학번:202120870 성명: 지민구	출 제 자	이두호
			채점성적	

1번

머신러닝2_1번 문제.R

```r
library(caret)
library(nnet)
library(dplyr)
load("P1.RData")

# 1-1
model_logit <- multinom(y ~ x1 + x2 + x3, data = P1_train)  # 다중 로지스틱 회귀 모델 학습
pred_logit <- predict(model_logit, newdata = P1_test, type = "class")  # 클래스 예측
test_y <- P1_test$y  # 테스트 데이터의 실제 정답값

acc_logit <- sum(pred_logit == test_y) / length(test_y)  # 정확도 계산
cat("[로지스틱 회귀] 분류 정확도:", sprintf("%.2f%%", acc_logit * 100), "\n")

# 1-2
X_train <- as.matrix(P1_train[, 1:3])  # 학습 데이터 행렬 변환
X_test <- as.matrix(P1_test[, 1:3])  # 테스트 데이터 행렬 변환
label_train <- P1_train$y  # 학습 데이터 정답 레이블

S <- cov(X_train)  # 공분산 행렬 계산
mahal_dist <- function(a, b) { sqrt(t(a - b) %*% solve(S) %*% (a - b)) }  # 마할라노비스 거리 함수

k_val <- 17  # 이웃 개수 설정
pred_knn <- sapply(1:nrow(X_test), function(idx) {
  query_pt <- X_test[idx, ]  # 현재 테스트 샘플
  all_dist <- apply(X_train, 1, function(row) mahal_dist(query_pt, row))  # 모든 학습 데이터와의 거리 계산
  top_k_labels <- label_train[order(all_dist)[1:k_val]]  # 가장 가까운 k개 이웃의 레이블 추출
  names(sort(table(top_k_labels), decreasing = TRUE))[1]  # 다수결로 예측값 결정
})

acc_knn <- sum(pred_knn == test_y) / length(test_y)  # KNN 정확도 계산
cat("[마할라노비스 KNN] 분류 정확도:", sprintf("%.2f%%", acc_knn * 100), "\n")
```

1-1

```
> cat("[로지스틱 회귀] 분류 정확도:", sprintf("%.2f%%", acc_logit * 100), "\n")
[로지스틱 회귀] 분류 정확도: 69.10%
```

1-2

```
> cat("[마할라노비스 KNN] 분류 정확도:", sprintf("%.2f%%", acc_knn * 100), "\n")
[마할라노비스 KNN] 분류 정확도: 68.75%
```

● 코드 흐름도 및 설명

- 다중 로지스틱 회귀 모델

* 다중 로지스틱 회귀 모델을 사용하여 3개 독립변수로 종속변수 y를 예측

1. multinom() 함수로 다항 로지스틱 회귀 모델을 학습
2. predict() 함수로 테스트 데이터의 클래스를 예측
3. 정확도 계산 후 출력

- 마할라노비스 거리 KNN

* 마할라노비스 거리 기반 KNN 분류 모델을 구현
공분산 행렬의 역행렬을 활용하여 상관관계를 고려한 거리를 계산
k=17의 최근접 이웃을 탐색하여 다수결 투표로 예측

1. 학습, 테스트 데이터를 행렬로 변환
2. 공분산 행렬 계싼
3. 마할라노비스 거리 공식 적용 후 함수 정의
4. 각 테스트 샘플에 대해 k=17개의 최근접 이웃 탐색
5. 다수결 투표로 예측
6. 정확도 계산 후 출력

● 결과 분석

로지스틱과 마할라노비스의 두 모델의 정확도를 비교한 결과 로지스틱 모델이 마할라노비스 모델보다 높은 분류 성능을 보였으며, 변수 간의 상관관계를 고려한 거리 측정이 분류 정확도에 긍정적으로 영향을 미쳤기 때문으로 판단.

번호 : /

강원대학교 시험 답안지

			감독교수확인	이 두 호
2025 학년도 2학기 기말고사	시험과목 : 머신러닝2 (3반)	전공 : AI소프트웨어학과 학번:202120870 성명: 지민구	출 제 자	이두호
			채점성적	

2번

```r
library(e1071)
library(caret)

# 데이터 로드
load("P2.RData")

# 학습 데이터 생성
train_data <- data.frame( # 학습 데이터프레임 생성
  x1 = c(-3, 3, -3, 3, 0, -1, 1, -1, 1),
  x2 = c(3, 3, -3, -3, 0, 1, 1, -1, -1),
  y  = factor(c("red", "red", "red", "red", "red", "blue", "blue", "blue", "blue"))
)

# 커널 함수 변환을 위한 함수 정의
add_kernel <- function(df) { # 커널 특성 추가 함수 정의
  df$f1 <- df$x1 # 원본 x1 특성
  df$f2 <- df$x2 # 원본 x2 특성
  df$f3 <- df$x1^2 # x1 제곱 특성
  df$f4 <- df$x2^2 # x2 제곱 특성
  df$f5 <- df$x1 * df$x2 # x1과 x2의 곱 특성
  df$f6 <- df$x1^2 + df$x2^2 # x1 제곱 + x2 제곱 특성
  return(df) # 변환된 데이터프레임 반환
}

# 데이터 변환 적용
train_mapped <- add_kernel(train_data) # 학습 데이터에 커널 변환 적용
test_mapped <- add_kernel(P2_test) # 테스트 데이터에 커널 변환 적용

# SVM 모델 학습
svm_model <- svm(y ~ f1 + f2 + f3 + f4 + f5 + f6, # SVM 모델 학습
  data = train_mapped, # 커널 변환된 학습 데이터
  kernel = "linear", # 선형 커널 사용
  cost = 1, # 비용 파라미터
  scale = FALSE # 데이터 정규화 비활성화
)
```

```r
# 예측 및 혼동행렬 생성
y_hat <- predict(svm_model, newdata = test_mapped) # 예측값 (Class 분류)
y_test <- test_mapped$y # 검증 데이터의 실제 정답값(Y)

# 결과 출력
conf_mat <- confusionMatrix(y_hat, y_test) # 혼동행렬 생성
conf_mat
print(conf_mat$table) # 혼동행렬 출력
```

<혼동행렬 출력값>

```
> print(conf_mat$table) # 혼동행렬 출력
          Reference
Prediction blue red
      blue   60  15
      red     0  25

           Accuracy : 0.85
             95% CI : (0.7647, 0.9135)
No Information Rate : 0.6
P-Value [Acc > NIR] : 5.073e-08

              Kappa : 0.6667
```

정확도 : (60+25) / 100 = 85%

● 코드 흐름도 및 설명

* 1사분면 -> FP - 실제 red 관측치 중 15개로 blue 잘못 예측
2사분면 -> TP - 실제 blue 관측치 중 60개 모두 blue 예측 성공
3사분면 -> FN - 실제 blue 관측치 중 0개로 red 잘못 예측
4사분면 -> TN - 실제 red 관측치 중 25개 모두 red 예측 성공

1. 9개의 샘플로 구성된 학습 데이터 생성(red : 5개, blue : 4개)
2. 원본 2차원 데이터를 6차원 특성 공간으로 확장
3. 선형 커널 SVM으로 분류 모델 학습
4. 테스트 데이터에 대해 예측 후 혼동행렬 출력

● 결과 분석

- 2차원에서 선형 분리가 불가능한 데이터를 커널 함수를 통해 6차원 공간으로 매핑한 후 SVM을 적용하여 분류를 수행함 분류를 통해 혼동행렬 결과를 통해 모델의 분류 성능을 확인할 수 있음
- 모델은 blue 클래스를 정확하게 분류하였지만 red 클래슨느 15개를 blue로 잘못 분류하여 전체 정확도 85%로 나온 것을 확인할 수 있음

번호 : /

강원대학교 시험 답안지

		감독교수확인	이 두 호

2025 학년도 2학기 기말고사	시험과목 : 머신러닝2 (3반)	전공 : AI소프트웨어학과 학번:202120870 성명: 지민구	출 제 자	이두호
			채점성적	

3번

```r
library(rpart)
library(rpart.plot)

# 1. 데이터 로드
load("P3.RData")

# 3-1. 선형회귀분석 - 절편 제거

# formula에 '-1'을 추가하여 절편을 강제로 0으로 만듦
lm_model <- lm(y ~ . - 1, data = P3_train) # 선형회귀 모델 학습

# 예측 수행
y_hat_lm <- predict(lm_model, newdata = P3_test) # 예측값 계산

# 평균제곱오차(MSE) 계산
mse_lm <- mean((P3_test$y - y_hat_lm)^2) # MSE 계산

cat("1. 선형회귀분석 모델 MSE:", mse_lm, "\n")

```

```r
# 3-2. 의사결정나무

ctrl <- rpart.control(cp = 0.013) # 복잡도 파라미터 설정

# 모델 학습
dt_model <- rpart(y ~ ., # 의사결정나무 모델 학습
    data = P3_train, # 학습 데이터
    method = "anova", # 회귀 문제용 분산 분석 (수치 예측이므로 method="anova")
    control = ctrl
) # 제어 파라미터

# 트리가 문제의 그림과 같은지 확인
rpart.plot(dt_model) # 의사결정나무 시각화

# 예측 수행
y_hat_dt <- predict(dt_model, newdata = P3_test) # 예측값 계산

# 평균제곱오차(MSE) 계산
mse_dt <- mean((P3_test$y - y_hat_dt)^2) # MSE 계산

cat("2. 의사결정나무 모델 MSE:", mse_dt, "\n")
```

3-1

```
> cat("1. 선형회귀분석 모델 MSE:", mse_lm, "\n")
1. 선형회귀분석 모델 MSE: 9938.556
```

3-2

```
> cat("2. 의사결정나무 모델 MSE:", mse_dt, "\n")
2. 의사결정나무 모델 MSE: 15739.95
```

● 코드 흐름도 및 설명

3-1

* $y = \sum_{i=1}^{6} w_i x_i$ 이 문제 조건에 따라 lm_model <- lm(y ~ . - 1, data = P3_train) 이 코드에서 -1을 추가하여 절편을 제거함 (회귀직선이 원점을 지나도록 강제함)

1. 절편을 제외한 선형회귀 모델 생성
2. 학습된 모델로 테스트 데이터 예측값 계산
3. 실제값과 예측값의 mse 계산
4. mse 값 출력

3-2

* 모델 학습을 통해 수치 예측이므로 anova를 사용 또는 cp = 0.013 복잡한 파라미터로 트리의 가지치기 수준을 조절

1. 복잡도 파라미터 cp = 0.013 설정
2. 회귀용 의사결정트리 생성 (method ="anova")
3. rpart.plot으로 트리 구조 확인
4. 테스트 데이터 예측값 계산 후 mse 계산 출력

● 결과 분석

두 모델을 비교한 결과 선형회귀모델의 mse가 의사결정나무 모델보다 약 43% 낮게 나타나 데이터 예측에 더 적합함을 확인함. 그래서 해당 데이터 셋이 비선형적 패턴보다 선형 관계를 따르는 특성을 가지고 있다고 판단.

번호 : /

강원대학교 시험 답안지

		감독교수확인	이 두 호	
2025 학년도 2학기 기말고사	시험과목 : 머신러닝2 (3반)	전공 : AI소프트웨어학과 학번:202120870 성명: 지민구	출 제 자	이두호
			채점성적	

4번

머신러닝2_2번 문제.R | Untitled1* | 머신러닝2_3번 문제.R | 머신러닝2_4번 문제.R | Untitled2*

Source on Save | Run | Source

```r
library(randomForest)
library(caret)

# 1. 데이터 로드
load("P4.RData")
set.seed(1000) # 시드 고정

# 4-1
rf_model <- randomForest(MedHouseVal ~ ., # 랜덤 포레스트 모델 학습
    data = P4_train, # 학습 데이터
    ntree = 1000, # 생성할 나무 개수
    mtry = 3, # 각 분기에서 고려할 변수 개수
    importance = TRUE, # 변수 중요도 계산
    do.trace = 100
) # 100 나무마다 진행상황 출력

# 결정계수 (R^2) 추출
r_squared <- tail(rf_model$rsq, 1) # 마지막 R^2 값 추출

# 결과 출력 (소수점 8자리까지 표현)
cat(sprintf("(1) 랜덤 포레스트 모델의 결정계수 (R^2): %.8f\n", r_squared))
```

```r
# 4-2
#  예측 및 MSE 계산
# 예측 수행
y_hat_rf <- predict(rf_model, newdata = P4_test) # 예측값 계산
# MSE 계산: mean((실제값 - 예측값)^2)
mse_rf <- mean((P4_test$MedHouseVal - y_hat_rf)^2) # 평균제곱오차 계산
cat("(2) P4_test 데이터에 대한 MSE:", mse_rf, "\n")

# 4-3. 변수 중요도 시각화
varImpPlot(rf_model, main = "Variable Importance (Random Forest)") # 변수 중요도
```

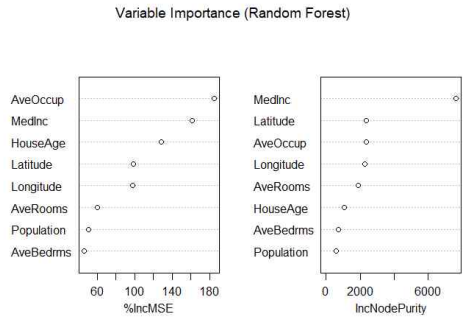
4-1

```
> cat(sprintf("(1) 랜덤 포레스트 모델의 결정계수 (R^2): %.8f\n", r_squared))
(1) 랜덤 포레스트 모델의 결정계수 (R^2): 0.81535691
```

4-2

```
> cat("(2) P4_test 데이터에 대한 MSE:", mse_rf, "\n")
(2) P4_test 데이터에 대한 MSE: 0.2288999
```

4-3



● 코드 흐름도 및 설명

* 랜덤포레스트 모델의 1000개의 의사결정트리를 생성, 각 분기에서 3개의 변수를 무작위로 선택하여 학습한 뒤 개별 트리의 예측을 평균내어 최종 예측값을 산출하는 앙상블 기법을 사용함

1. 데이터 로드 및 시드 고정(set.seed(1000))
2. 랜덤 포레스트 모델 생성 (ntree = 1000, mtry =3)
3. 학습된 모델에서 결정계수 추출
4. 테스트 데이터에 대한 예측 후 mse 계산
5. varImPlot으로 변수 중요도 그래프 출력

● 결과 분석

랜덤포레스트 모델을 사용하여 학습한 결과 결정계수는 약 0.815로 나왔으며 테스트 데이터에 대한 mse는 약 0.229로 측정됨.
변수 중요도 분석에 %IncMSE 기준으로는 AveOccup이 IncNodePurity 기준으로는 MedInc이 가장 중요한 변수로 판단.
변수 중요도 분석 중에 Population과 AveBedrms의 두 기준 모두 상대적으로 낮은 중요도를 보여 주택 가격 예측에 미치는 영향이 적다는 것으로 판단.

번호 : /

강원대학교 시험 답안지

			감독교수확인	이 두 호
2025 학년도 2학기 기말고사	시험과목 : 머신러닝2 (3반)	전공 : AI소프트웨어학과 학번:202120870 성명: 지민구	출 제 자	이두호
			채점성적	

5번

머신러닝2_1번 문제.R | 머신러닝2_2번 문제.R | Untitled1* | 머신러닝2_3번 문제.R | 머신러닝2_4번 문제.R | 머신러닝2_5번 문제.R

Source on Save Run

```r
# 데이터 정의

# 입력변수 x1, x2
input_data <- matrix(
  c(
    1.2, 2.5, # 관측치 A
    2.0, 1.0, # 관측치 B
    -1.5, -2.0, # 관측치 C
    -2.2, -1.0, # 관측치 D
    0.5, -1.0
  ), # 관측치 E
  ncol = 2, byrow = TRUE
) # 5x2 행렬로 변환

# 출력변수 y (Red = 1, Blue = -1)
# 문제 조건: w*x <= 0 이면 Blue. Blue는 음수 영역 Red는 양수 영역
output_data <- c(1, 1, -1, -1, 1) # 각 관측치의 레이블

# 초기값 설정
weight <- c(0, 0) # 초기 가중치
gamma <- 1 # 마진 값

# 마진 퍼셉트론 업데이트 함수 정의
update_perceptron <- function(w, x, y, gamma) { # 퍼셉트론 업데이트 함수
  # 현재 가중치로 점수 계산 (Dot Product)
  score <- sum(w * x) # 내적 계산

  # 업데이트 조건: y * (w * x) < gamma 인 경우 (마진보다 작으면 업데이트)
  if ((y * score) < gamma) { # 업데이트 필요 여부 확인
    # tau 계산: (gamma - y * score) / ||x||^2
    norm_sq <- sum(x^2) # x의 L2 norm 제곱 계산
    tau <- (gamma - (y * score)) / norm_sq # 학습률 계산

    w_new <- w + (tau * y * x) # 새로운 가중치 계산 (가중치 업데이트: w_new = w + tau * y * x)
    return(w_new) # 업데이트된 가중치 반환
  }

  # 조건을 만족하면 업데이트 없이 그대로 반환
  return(w) # 기존 가중치 반환
}

# 5-1 관측치 A 업데이트

# A는 첫 번째 데이터 (row 1)
w1 <- update_perceptron(weight, input_data[1, ], output_data[1], gamma) # 관측치 A로 가중치 업데이트
cat("(1) 관측치 A 업데이트 후 w(1):", w1, "\n")

# 5-2  관측치 B 업데이트
# 이어서 관측치 B에 대해 업데이트 된 w(2) 구하기

# w1 상태에서 B(row 2)를 넣어 업데이트
w2 <- update_perceptron(w1, input_data[2, ], output_data[2], gamma) # 관측치 B로 가중치 업데이트
cat("(2) 관측치 B 업데이트 후 w(2):", w2, "\n")

# 5-3. 1 Epoch (A -> E) 학습
# 1 Epoch (A -> E) 학습 후 w(5) 구하기

# 현재 w2 상태에서 나머지 C, D, E를 순서대로 학습
w_curr <- w2 # 현재 가중치를 w2로 초기화
for (i in 3:5) { # 관측치 C, D, E 반복
  w_curr <- update_perceptron(w_curr, input_data[i, ], output_data[i], gamma) # 순차적으로 업데이트
}
w5 <- w_curr # 1 Epoch 종료 후 가중치 저장
cat("(3) 1 Epoch (A~E) 종료 후 w(5):", w5, "\n")

# 5-4. 200 Epoch 학습
# 200 Epoch 학습 후 w(1000) 구하기

# 처음(0,0)부터 다시 시작해서 200바퀴 돌아감
w_final <- c(0, 0) # 가중치 초기화

for (epoch in 1:200) { # 200 Epoch 반복
  for (i in 1:5) { # 1 Epoch 당 5개의 데이터(A~E) 순회
    w_final <- update_perceptron(w_final, input_data[i, ], output_data[i], gamma) # 순차적으로 업데이트
  }
}

cat("(4) 200 Epoch 종료 후 w(1000):", w_final, "\n")
```

번호 : /

강원대학교 시험 답안지

		감독교수확인	이 두 호	
2025 학년도 2학기 기말고사	시험과목 : 머신러닝2 (3반)	전공 : AI소프트웨어학과 학번:202120870 성명: 지민구	출 제 자	이두호
			채점성적	

5번

5-1

```
> cat("(1) 관측치 A 업데이트 후 w(1):", w1, "\n")
(1) 관측치 A 업데이트 후 w(1): 0.1560468 0.3250975
```

5-2

```
> cat("(2) 관측치 B 업데이트 후 w(2):", w2, "\n")
(2) 관측치 B 업데이트 후 w(2): 0.3011704 0.3976593
```

5-3

```
> cat("(3) 1 Epoch (A~E) 종료 후 w(5):", w5, "\n")
(3) 1 Epoch (A~E) 종료 후 w(5): 0.8 -0.6
```

5-4

```
> cat("(4) 200 Epoch 종료 후 w(1000):", w_final, "\n")
(4) 200 Epoch 종료 후 w(1000): 1.428571 -0.2857143
```

● 코드 흐름도 및 설명

* 마진 퍼셉트론을 활용하여 일반 퍼셉트론과 달리 단순 분류 정확도가 아닌 결정 경계로부터의 여유 공간을 최대화 하는 방향으로 학습, 예측값과 실제값의 곱이 설정된 마진보다 작으면 가중치를 조정하여 더 안정적인 분류 경계를 형성시킴

1. 5개의 관측치 입력 데이터와 Red = 1, Blue = 1 설정
2. 마진 조건을 만족하지 않으면 가중치를 업데이트하는 함수 구현 (퍼셉트론 함수)
3. 초기 가중치 (0,0)에서 관측치 A로 w1 계산
4. w1 상태에서 관측치 B로 w2 계산
5. A ~ E 전체를 순회하여 w5 계산
6. 전체 데이터를 200번 반복하여 w1000 계산

● 결과 분석

마진 퍼셉트론을 활용하여 5개의 2차원 데이터를 두 클래스로 분류하는 과정을 단계별로 구현한 결과 학습 초기에는 가중치가 빠르게 변환하며 w1(0.156, 0.325) -> w2(0.301, 0.398) -> w5(0.8,-0.6)으로 조정됨.
200회 Epoch 학습 후 가중치가 약 w1000(1.429, -0.286)으로 수렴함에 따라 모델이 안정된 분류 경계를 학습한 것으로 확인할 수 있음. 즉 충분한 반복 학습을 통해 마진 조건을 만족하는 최적의 가중치가 나온다고 판단.

번호 : /

강원대학교 시험 답안지

			감독교수확인	이 두 호
2025 학년도 2학기 기말고사	시험과목 : 머신러닝2 (3반)	전공 : AI소프트웨어학과 학번:202120870 성명: 지민구	출 제 자	이두호
			채점성적	

6번

```r
rm(list = ls())

# 6-1. 선형회귀 모델
load("P6(1).RData")

train_df <- na.omit(P6_train) # 결측치 제거
reg_model <- lm(medv ~ ., data = train_df) # 선형회귀 모델 학습
cat("선형회귀 R^2:", summary(reg_model)$r.squared, "\n") # 결정계수 출력

# 6-2. 인공신경망

# 정규화 함수 정의
normalize_data <- function(data) { # Min-Max 정규화 함수
  max_v <- apply(data, 2, max) # 각 열 최댓값
  min_v <- apply(data, 2, min) # 각 열 최솟값
  normalized <- scale(data, center = min_v, scale = max_v - min_v) # 정규화 수행
  list(data = as.data.frame(normalized), max = max_v, min = min_v) # 결과 반환
}

# 순전파 함수
forward_pass <- function(input, weights, biases, act_func) { # 순전파 계산 함수
  n_layers <- length(weights) # 레이어 수
  activations <- list() # 활성화 값 저장
  z_values <- list() # 가중합 저장
  current_input <- input # 현재 입력
```

```r
  }
  list(out = activations[[n_layers]], acts = activations, zs = z_values) # 결과 반환
}

# 데이터 준비
norm_result <- normalize_data(train_df) # 정규화 수행
scaled_df <- norm_result$data # 정규화된 데이터
features <- as.matrix(scaled_df[, names(scaled_df) != "medv"]) # 입력 특성 추출
target <- as.matrix(scaled_df$medv) # 출력값 추출

# 활성함수 정의
linear_act <- function(x) 1.7 * x # f(x) = 1.7x
linear_act_grad <- function(x) 1.7 # f'(x) = 1.7

# 네트워크 구성
layer_sizes <- c(ncol(features), 5, 5, 1) # 입력-은닉1-은닉2-출력 노드 수
eta <- 0.34 # 학습률
total_iter <- 20000 # 총 반복 횟수
N <- nrow(features) # 샘플 수
```

```r
# 가중치/편향 초기화
set.seed(1234) # 재현성을 위한 시드 고정
wts <- list(); bs <- list() # 가중치, 편향 리스트
for (k in 1:(length(layer_sizes) - 1)) { # 각 레이어 연결
  wts[[k]] <- matrix(rnorm(layer_sizes[k] * layer_sizes[k+1], 0, 0.001),
                     layer_sizes[k], layer_sizes[k+1]) # 가중치 초기화
  bs[[k]] <- matrix(0, 1, layer_sizes[k+1]) # 편향 초기화
}

# 클리핑 함수
bound_grad <- function(x) pmax(pmin(x, 5), -5) # 그래디언트 클리핑
```

번호 : /

강원대학교 시험 답안지

		감독교수확인	이 두 호	
2025 학년도 2학기 기말고사	시험과목 : 머신러닝2 (3반)	전공 : AI소프트웨어학과 학번:202120870 성명: 지민구	출 제 자	이두호
			채점성적	

6번

```
# 학습 루프
for (iter in 1:total_iter) { # 반복 학습
  # 순전파
  fwd <- forward_pass(features, wts, bs, linear_act) # 순전파 수행
  output <- fwd$out # 출력값
  hidden_acts <- fwd$acts # 은닉층 활성화
  hidden_zs <- fwd$zs # 은닉층 가중합

  # 역전파 (출력층)
  delta <- (output - target) / N # 오차 계산
  grad_w <- list(); grad_b <- list() # 그래디언트 저장

  # 레이어 3 (출력층)
  prev_act <- hidden_acts[[2]] # 이전 층 활성화
  grad_w[[3]] <- bound_grad(t(prev_act) %*% delta) # 가중치 그래디언트
  grad_b[[3]] <- bound_grad(colSums(delta)) # 편향 그래디언트

  # 레이어 2
  delta <- (delta %*% t(wts[[3]])) * linear_act_grad(hidden_zs[[2]]) # 델타 역전파
  prev_act <- hidden_acts[[1]] # 이전 층 활성화
  grad_w[[2]] <- bound_grad(t(prev_act) %*% delta) # 가중치 그래디언트
  grad_b[[2]] <- bound_grad(colSums(delta)) # 편향 그래디언트

  # 레이어 1
  delta <- (delta %*% t(wts[[2]])) * linear_act_grad(hidden_zs[[1]]) # 델타 역전파
  grad_w[[1]] <- bound_grad(t(features) %*% delta) # 가중치 그래디언트
  grad_b[[1]] <- bound_grad(colSums(delta)) # 편향 그래디언트

  # 파라미터 갱신
  for (k in 1:3) { # 각 레이어에 대해
    wts[[k]] <- wts[[k]] - eta * grad_w[[k]] # 가중치 업데이트
    bs[[k]] <- bs[[k]] - eta * grad_b[[k]] # 편향 업데이트
  }
}
```

```
# 최종 예측
final_pred <- forward_pass(features, wts, bs, linear_act)$out # 최종 순전파

# 역정규화 및 R² 계산
pred_original <- final_pred * (norm_result$max["medv"] - norm_result$min["medv"]) + norm_result$min["medv"] # 역정규화
actual_vals <- train_df$medv # 실제값

ss_res <- sum((actual_vals - pred_original)^2) # 잔차제곱합
ss_tot <- sum((actual_vals - mean(actual_vals))^2) # 총제곱합
r2_neural <- 1 - ss_res / ss_tot # 결정계수 계산

cat("인공신경망 R^2:", r2_neural, "\n") # 결과 출력
```

6-1

```
> cat("선형회귀 모델 결정계수 (R^2):", summary(lm_model)$r.squared, "\n")
선형회귀 모델 결정계수 (R^2): 0.7562637
```

6-2

```
> cat("인공신경망 모델 결정계수 (R^2):", 1 - (SSE / SST), "\n")
인공신경망 모델 결정계수 (R^2): 0.7562637
```

번호 : /

강원대학교 시험 답안지

		감독교수확인	이 두 호	
2025 학년도 2학기 기말고사	시험과목 : 머신러닝2 (3반)	전공 : AI소프트웨어학과 학번:202120870 성명: 지민구	출 제 자	이두호
			채점성적	

6번

● 코드 흐름도 및 설명

* 6-1 선형회귀 모델

선형회귀모델을 통해 모든 독립변수를 사용하여 주택 가격을 예측하는 선형회귀 모델을 학습 시킨 후 결정계수 추출

* 6-2 인공신경망

인공신경망은 13개 입력 변수를 받아 두 개의 은닉층을 거쳐 주택 가격을 예측하며 학습 안정성을 위해 Min-Max 정규화를 적용함.

활성함수 f(x)=1.7x를 사용하여 순전파와 역전파를 수행하고 그래디언트 클리핑으로 오류 방지하면서 학습률 0.34로 20000회 학습

1. 선형회귀모델을 학습하기 위해서 lm()을 사용해야 하기 때문에 직접 구현한 신경망에서 na가 있으면 행렬 연산이 불가능하기 때문에 결측치 제거
2. 기본 선형회귀 모델로 결정 계수 계산
3. 변수들의 범위가 다르면 신경망 학습이 불안정하기 때문에 Min-Max 스케일링 함수 구현
4. 다층 신경망의 순전파 계산 함수 구현
5. 은닉층 2개(각 5노드), 활성함수 f(x)=1.7x 설정
6. 작은 랜덤값으로 가중치 초기화
7. 20000번 반복하여 순전파 -> 역전파 -> 가중치 갱신
8. 역정규화 후 결정 계수 계산

● 결과 분석

선형회귀모델의 결정계수는 약 0.756으로 주택 가격 변동을 설명할 수 있음을 확인할 수 있음. 인공신경망 모델은 은닉층 2개, 활성함수 f(x) = 1.7x, 학습률 0.34로 20000회 학습한 결과 선형회귀와 동일한 결정계수 나온 것을 확인함.

해당 데이터가 선형적 관계를 따르는 특성을 가지고 있어 비선형 모델인 인공신경망이 선형회귀 대비 큰 성능 향상을 보이지 않는다고 판단.

● 추가 내용

6번 문제를 풀기 위해 처음에 사용했던 코드는 neuralnet 라이브러리를 사용해서 rprop- 알고리즘을 사용함. 반복적인 오류로 인해 AI툴로 통해 조사한 결과 rprop- 알고리즘은 학습률 파라미터를 내부적으로 무시하여 문제에서 요구하는 학습률 0.34를 정확히 반영할 수 없다고 판단. 그래서 직접 인공신경망을 구현하여 학습률이 가중치 업데이트에 적용되어 문제 조건을 충족할 수 있었음.

번호 : /

강원대학교 시험 답안지

			감독교수확인	이 두 호
2025 학년도 2학기 기말고사	시험과목 : 머신러닝2 (3반)	전공 : AI소프트웨어학과 학번:202120870 성명: 지민구	출 제 자	이두호
			채점성적	

7번

제.R × | 머신러닝2_2번 문제.R × | 6번_test.R × | 머신러닝2_3번 문제.R × | 머신러닝2_4번 문제.R

Source on Save

```r
# 초기 가중치 및 데이터 설정

# w.1: 은닉층 가중치
w_1 <- matrix(c(7, 0.7,
                0.78, -2.5), nrow = 2, byrow = TRUE)

w_2 <- matrix(c(-2, 7), nrow = 1) # w.2: 출력층 가중치

bias_1 <- matrix(c(5, 2), nrow = 2) # b.1: 은닉층 편향


bias_2 <- matrix(6) # b.2: 출력층 편향

# 입력값(x)과 정답(y), 학습률(lr)
input_x <- matrix(c(3, 11.2), nrow = 2) # 입력 데이터
y_target <- 0.27 # 목표 출력값
learning_rate <- 0.18 # 학습률

# 활성화 함수 정의

# 은닉층: tanh
# tanh 활성화 함수
# 출력층: sigmoid
activation_1 <- function(z) {
  tanh(z)
}

# sigmoid 활성화 함수
activation_2 <- function(z) {
  1 / (1 + exp(-z))
}

# 오차역전파 학습 (315회 반복)

for (i in 1:315) { # 315회 반복 학습
  # 순전파
  z1 <- (w_1 %*% input_x) + bias_1 # 은닉층 입력값 계산
  a1 <- activation_1(z1) # 은닉층 출력 (tanh 적용)

  z2 <- (w_2 %*% a1) + bias_2 # 출력층 입력값 계산
  y_hat <- activation_2(z2) # 출력층 예측값 (sigmoid 적용)

  # 역전파
  # sigmoid 미분 = y_hat * (1 - y_hat)
  delta_2 <- (y_hat - y_target) * (y_hat * (1 - y_hat)) # 출력층 오차 계산

  # 은닉층 오차
  # tanh 미분 = 1 - tanh^2 = 1 - a1^2
  delta_1 <- (t(w_2) %*% delta_2) * (1 - a1^2) # 은닉층 오차 계산

  # 가중치 갱신
  w_2 <- w_2 - learning_rate * (delta_2 %*% t(a1)) # 출력층 가중치 갱신
  bias_2 <- bias_2 - learning_rate * delta_2 # 출력층 편향 갱신

  w_1 <- w_1 - learning_rate * (delta_1 %*% t(input_x)) # 은닉층 가중치 갱신
  bias_1 <- bias_1 - learning_rate * delta_1 # 은닉층 편향 갱신
}

# 결과 출력
cat("315회 학습 후 출력값(y_hat):", y_hat, "\n")
```

7

```
> cat("315회 학습 후 출력값(y_hat):", y_hat, "\n")
315회 학습 후 출력값(y_hat): 0.2565109
```

번호 : /

강원대학교 시험 답안지

		감독교수확인	이 두 호	
2025 학년도 2학기 기말고사	시험과목 : 머신러닝2 (3반)	전공 : AI소프트웨어학과 학번:202120870 성명: 지민구	출 제 자	이두호
			채점성적	

7번

● 코드 흐름도 및 설명

* 2층 인공신경망에 오차역전파 알고리즘을 적용하여 가중치를 갱신하는 과정 구현
* 구조는 2개의 입력 노드(x1, x2), 2개의 은닉 노드, 1개의 출력 노드로 구성
* 은닉층은 tanh 활성함수 사용, 출력층은 sigmoid 활성함수 사용
* 순전파 과정에서 입력값에 가중치를 곱하고 편향을 더한 뒤 활성함수를 통과시켜 예측값을 받음
* 역전파 과정에서 예측값과 실제값의 오차를 계산하고 체인 룰을 적용시켜 각 가중치의 그레디언트를 구함 (sigmoid 미분은 y(1-y), tanh 미분은 1-tanh^2(z)

1. 가중치, 편향, 입력값, 목표값, 학습률 설정
2. 은닉층(tanh), 출력층(sigmoid) 함수 정의
3. 315회 반복하며 순전파 -> 역전파 -> 가중치 갱신
4. 학습 수 예측값 출력

● 결과 분석

2층 인공신경망에 입력값 (3,11.2)와 목표값 0.27을 설정하고 학습률 0.18로 오차역전파 알고리즘을 315회 수행함.
은닉층에는 tanh 활성함수, 출력층에는 sigmoid 활성함수를 적용하여 순전파를 수행하고 각 활성함수의 미분값을 이용해 역전파로 그레디언트를 계산.
315회 반복 학습 결과, 최종 예측값은 약 0.2565109로 목표값 0.27에 근접하게 수렴한 것으로 확인.
그 결과 오차역전파 알고리즘이 가중치를 효과적으로 갱신하여 예측 정확도를 향상 시킴을 확인함.

번호 : /

강원대학교 시험 답안지

		감독교수확인	이 두 호	
2025 학년도 2학기 기말고사	시험과목 : 머신러닝2 (3반)	전공 : AI소프트웨어학과 학번:202120870 성명: 지민구	출 제 자	이두호
			채점성적	

8번

저는 올해 겨울방학 기간동안 연구실에서 학부연구생으로 일을 하며 학교 친구들과 창업 준비를 할 예정입니다. 창업 아이디어 구상부터 시작하여 그 아이디어를 직접 개발하고 구현할 것입니다. 그 후 창업 공모전도 준비해서 성공과 동시에 실패도 겪어볼려고 합니다. 창업 아이디어를 구상할 때 클라우드 기술을 활용할 것이며 그것을 통해 클라우드 기술도 공부를 할 예정입니다. 추가로 토익 공부와 클라우드 자격증인 AWS도 공부할 예정입니다. 그리고 개강하기 전에 학교 친구들과 학기 중에 시험으로 인한 스트레스를 풀기 위해 2월에 스키장을 가서 스트레스를 해소 할 예정입니다.